



UNIVERSITI MALAYA

KOD KURSUS	:	WIX1002
NAMA KURSUS	:	FUNDAMENTALS OF PROGRAMMING
NAMA PENSYARAH	:	DR UNAIZAH HANUM BINTI OBAIDELLAH
SEMESTER & SESI	:	SEMESTER 1, 2024/2025
KUMPULAN	:	OCC 1

BIL.	NAMA	NO MATRIK
1	AHMAD MUKHLIS BIN MOHD NORDIN	23002476
2	AZNUR ANAK JITWIL @ JIKUIL	23001792
3	MUHAMMAD AFIQ BIN MOHD FAUZI	23002465
4	MUHAMMAD SYAMEER HARITH BIN MOHD YUSOF	23002196
5	JOECHELE LIM QIU YING	24004535

Contents

QUESTION 1	1
Problem.....	1
Solution	1
Pseudocode	2
Sample Input and Output.....	2
Source code.....	3
QUESTION 2	8
Problem.....	8
Solution	8
Pseudocode	10
Sample Input and Output.....	12
Source Code.....	13
QUESTION 3: Romans	17
Problem Statement	17
Solution	18
Pseudocode	21
Sample input and output	25
Source Code.....	26
QUESTION 4	32
Problem Statement	32
Solution	32
Pseudocode	33
Sample input and output	34
Source Code.....	35
QUESTION 5	37

Problem.....	37
Solution	38
Pseudocode	40
Sample Input and Output.....	42
Source Code.....	44
QUESTION 6	47
Problem.....	47
Solution	48
Pseudocode	48
Sample input and output	49
Source code	51

QUESTION 1

Problem

Write a method called ***findLongestPalindromicSubstring*** to find the longest palindromic substring in a given string. A *palindromic substring* is a substring of the given string that reads the same forwards and backwards. The method should return the longest palindromic substring found within the input string.

In the ***main*** method, get input from the user and then display the longest palindromic substring within the input string using the ***findLongestPalindromicSubstring***

Solution

Palindromic is a word that can read both ways, hence a letter is basically a palindromic too.

Letter 'A' can be read both ways. My approach to answer this question is by using Boolean 2D array. To better understand the method, I will use the table.

	M	A	D	A	M
M	T	F	F	F	F
A	F	T	F	F	F
D	F	F	T	F	F
A	F	F	F	T	F
M	F	F	F	F	T

Because we are not supposed to start in the middle of the word for this problem, we need to check for each letter in the word.

The program will check the value between each TRUE. If the letter with TRUE value is sandwiched between the same word, the program will turn into TRUE.

For example, the 3rd row from the table will turn into this :

	M	A	D	A	M
D	F	T	T	T	F

And the program loop until they reach the border of the array.

However, a problem occur when words such as “ahha” appeared. There is no words that can become the middle word. Hence, the program needs to check whether the words is same or not.

	A	H	H	A
A	T	F	F	F
H	F	T	T	F
H	F	F	T	F
A	F	F	F	T

If the words is same, there will be 2 TRUE value, and the program will check the letter that sandwiched both TRUE values.

In the end, the program will print out the word from a row with the most TRUE value.

Pseudocode

FindLongestPalindromicSubstring (METHOD)

1. **RECIEVE** string from main
2. **CREATE** 2D Boolean array with size of the string and fill everything with false but fill the same index with true (Boolean[i][i])
3. **CHECK** if there is a same letter side by side from each row inside boolean
Comparison by using `.charAt[i] == .charAt[i+1]`. If same, Boolean for [i+1] will turn into true.
4. **CHECK** if the character squeezing the TRUE value is same. If same, both word Boolean will turn true . loop until there is no longer same letters, or until the border of the word is reached.
5. **COUNT** for the longest true value at which row
6. **PRINT** the row with the longest true value. Only print letters that are true
7. **END**

Main method

1. **READ** user input
2. **RUN** FindLongestPalindromicSubstring (input)

Sample Input and Output

```
run:
Enter string : neveroddoreven
neveroddoreven
BUILD SUCCESSFUL (total time: 6 seconds)
```

```
run:
Enter string : ahhaabcde
ahha
BUILD SUCCESSFUL (total time: 7 seconds)
```

```
run:
Enter string : absde
a
BUILD SUCCESSFUL (total time: 11 seconds)
```

Source code

```
import java.util.Scanner;

public class Q1 {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter string : ");

        String word = scanner.nextLine().toLowerCase();

        findLongestPalindromicSubstring(word);

    }

    public static void findLongestPalindromicSubstring(String word){

        boolean[][] check = new boolean[word.length()][word.length()];

        //fill up 2D boolean with false

        for(int i=0; i<word.length();i++){

            for(int j=0; j<word.length();j++){

                check[i][j] = false;

            }

            //all section with same index is true bcs they are one palindromic

            check[i][i]= true;

        }

    }

}
```

```

//if the letter is same side by side, it is palindromic

for(int i=0; i<word.length()-1; i++){

    if(word.charAt(i)==word.charAt(i+1) && i<word.length()-1){

        check[i][i+1] = true;

    }

}

```

```

//the final check

for(int i=1; i<word.length()-1;i++){

    int k=1;

    boolean a = true;

    for(int j=1; j<word.length()-1;j++){

        if(check[i][j]){

            if(check[i][j+1]){

                while(a){

                    if(word.charAt(j-k) == word.charAt(j+1+k)){

                        check[i][j-k] = true;

                        check[i][j+1+k] = true;

                    }

                    if(j==k || j+1+k == word.length()-1){

                        a=false;

                    }else{

```

```

        k++;

    }

}

}else{

    while(a){

        if(word.charAt(j-k) == word.charAt(j+k)){

            check[i][j-k] = true;

            check[i][j+k] = true;

        }

        if(word.length()-1 == (j+k) || (j-k) == 0){

            a=false;

        }else{

            k++;

        }

    }

}

}

}

//check the longest

int max=0;

int index=0;

int[] longest= new int[word.length()];

```



```

for(int i=0; i<word.length(); i++){
    for(int j=0; j<word.length(); j++){
        if(check[i][j])
            longest[i]+=1;
    }
    if(longest[i] >max){
        index = i;
        max = longest[i];
    }
}

//print the word
for(int j=0; j<word.length(); j++){
    if(check[index][j]){
        System.out.print(word.charAt(j));
    }
}

System.out.println();

//    System.out.println(index);
//    for(boolean[] arr : check){
//        for(boolean value : arr){
//            System.out.print(value + " ");
//        }
//    }
//    System.out.println();

```

```
//    }  
  
//    for(int a : longest){  
  
//        System.out.println(a);  
  
//    }  
  
    }  
  
}
```

QUESTION 2

Problem

Problem Statement Construct a Java program that receives two lines of strings from the user and interprets them as integer arrays. You should use a method called `parseArray`, which takes the input and returns the array. Then, construct another method called `mergeArray`, which combines the two arrays into one array, while sorting all integers in ascending order and removing any repeated values. Print the final merged array. You may construct any additional methods that may help you in solving the problem.

Sample Input	Sample Output
Array 1: 1, 2, 3 Array 2: 6, 3, 4	[1, 2, 3, 4, 6]
Array 1: 5, 2, 1, 3, 8 Array 2: 8, 1	[1, 2, 3, 5, 8]

Note: You are not allowed to use the `ArrayList`.

Solution

1. Import the `Scanner` class so the user can input two **Strings**: `array1` and `array2`.
2. After that, a method called “**`parseArray`**” is created to split **Strings** into **integer** arrays. The input string is separated using a **comma (,)** as the delimiter to form a string array. Each string element is converted into an integer and stored in an integer array. The resulting integer array is returned.
3. In addition, the **`mergeArray`** method combines two integer arrays, **`array1`** and **`array2`**, into a single merged array while ensuring no duplicates and sorting the result in ascending order.
A temporary array (**`tempArray`**) is created to hold all elements from both input arrays. A bubble sort is applied to the temporary array to arrange the elements in ascending order.
A new array (**`result`**) is created to hold only unique elements from the sorted temporary array. A variable (**`uniqueCount`**) keeps track of the count of unique elements. Duplicate elements are skipped during the transfer from the temporary array to the final array.
The **`result`** array is returned after deduplication and sorting.

4. In the main method, the user is prompted to input two strings: array1 and array2.
The **parseArray** method is used to convert the strings into integer arrays.
These arrays are passed to the **mergeArray** method to combine, deduplicate, and sort the integers.
The final sorted and merged array is printed.

Pseudocode

Main Method

Begin

```
display "Array 1: "  
read array1  
display "Array 2: "  
read array2  
display "The output is: "  
int[] intArray1 = parseArray(array1)  
int[] intArray2 = parseArray(array2)  
int[] finalArray = mergeArray(intArray1, intArray2);  
display "Merge Array: ["  
for (i = 0; i < finalArray.length; i++)  
    if (i == finalArray.length - 1)  
        display finalArray[i]  
    else  
        display finalArray[i] + ", "  
    end if  
end for  
display "]"
```

End

ParseArray(String line) Method

Begin

```
String[] stringArray1 = line.split(",")  
int[] intArray = new int[stringArray.length]  
for (i = 0; i < stringArray.length; i++)  
    intArray[i] = Integer.parseInt(stringArray[i]);  
end for  
return intArray
```

End

mergeArray(int[] intArray1, int[] intArray2) Method

Begin

```
size1 = intArray1.length
```

```
size2 = intArray2.length
```

```
int[] tempArray= new int[size1 + size2]
```

```
int[] result = new int[sizeArray[intArray1, intArray2]
```

```
index=0
```

```
for (int i = 0; i<size1; i++)
```

```
    tempArray[i] = intArray1[i]
```

```
end for
```

```
for (int i = 0; i<size2; i++)
```

```
    tempArray[size1 + i] = intArray2[i]
```

```
end for
```

```
temp
```

```
for( i=1; i< tempArray.length; i++)
```

```
    for(j =0; j< tempArray.length-1; j++)
```

```
        if(tempArray [j] > tempArray [j+1])
```

```
            temp = tempArray [j]
```

```
            tempArray [j] = tempArray [j+1]
```

```
            tempArray [j+1] = temp
```

```
        end if
```

```
    end for
```

```
end for
```

```
uniqueCount = 1
```

```
result[0] = tempArray[0]
```

```
index = 1
```

```
for (int i = 0; i<tempArray.length; i++)
```

```
    if(! (tempArray [i] == tempArray [i+1])) )
```

```
result[index++] = tempArray[i+1]
```

```
end if
```

```
end for
```

return result

End

Sample Input and Output

Example 1:

```
run:
Please enter two arrays with integers (comma-seperated).
Array 1: 1,2,3
Array 2: 6,4,3
Merge Array: [1, 2, 3, 4, 6]
BUILD SUCCESSFUL (total time: 6 seconds)
```

Example 2:

```
run:
Please enter two arrays with integers (comma-seperated).
Array 1: 5,2,1,3,8
Array 2: 8,1
Merge Array: [1, 2, 3, 5, 8]
BUILD SUCCESSFUL (total time: 9 seconds)
```

Example3:

```
run:
Please enter two arrays with integers (comma-seperated).
Array 1: 1,3,3
Array 2: 2,4,8,3
Merge Array: [1, 2, 3, 4, 8]
BUILD SUCCESSFUL (total time: 9 seconds)
```

Source Code

```
import java.util.Scanner;

public class Q2 {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.println("Please enter two arrays with integers (comma-seperated).");

        System.out.print("Array 1: ");

        String array1 = input.nextLine();

        System.out.print("Array 2: ");

        String array2 = input.nextLine();

        // Call array

        int[] intArray1 = parseArray(array1);

        int[] intArray2 = parseArray(array2);

        int[] finalArray = mergeArray(intArray1, intArray2);

        System.out.print("Merge Array: [");

        for (int i=0; i< finalArray.length; i++) {

            if (i == finalArray.length-1) {

                System.out.print(finalArray[i]);
```



```

        } else {

            System.out.print(finalArray[i] + ", ");

        }

    }

    System.out.println("");

    input.close();

}

public static int[] parseArray(String line) {

    // Split array when there are ","

    String[] stringArray = line.split(",");

    int[] intArray = new int[stringArray.length];

    // To input number in intArray from stringArray

    for (int i = 0; i < stringArray.length; i++) {

        intArray[i] = Integer.parseInt(stringArray[i]);

    }

    return intArray;

}

```

```

public static int[] mergeArray (int[] intArray1, int[] intArray2) {

    int size1 = intArray1.length;

    int size2 = intArray2.length;

    int[] tempArray = new int[size1 + size2];

    for(int i=0; i<size1; i++) {

        tempArray[i] = intArray1[i];

    }

    for(int i=0; i<size2; i++) {

        tempArray[size1 +i] = intArray2[i];

    }

    // To sort number in ascending

    int temp;

    for(int i=1; i<tempArray.length; i++) {

        for(int j =0; j<tempArray.length-1; j++) {

            if(tempArray[j] > tempArray[j+1]) {

                temp = tempArray[j];

                tempArray[j] = tempArray[j+1];

                tempArray[j+1] = temp;

            }

        }

    }

}

```

```

    }
}

// Calculate size without repeating element
int uniqueCount = 1;
for(int i=0; i<tempArray.length-1; i++) {
    if(!(tempArray[i] == tempArray[i+1])) {
        uniqueCount++;
    }
}

int[] result = new int[uniqueCount];
result[0] = tempArray[0]; // Frist element is always include
int index = 1; // Result array index
for(int i=0; i<tempArray.length-1; i++) {
    if(!(tempArray[i] == tempArray[i+1])) {
        result[index++] = tempArray[i+1];
    }
}

return result;
}
}

```

QUESTION 3:

Problem Statement

In the mid-lecture break of Computing Mathematics II, Kah Sing chatted about something that he found very interesting to his friends Ridwan and Suresh in the lecture hall.

Kah Sing: ‘Eh, Wan, Su, ever wondered why Dr Soh doesn’t name this Computing Mathematics module IIIV, IIIVX, IIIVXXXXL instead? There are so many ways to write it sia!’

Ridwan: ‘Haha, of course not, kan! Imagine memorising Roman characters this long just for the number “2”!’

Suresh: ‘Deyh, you’re right, but actually hor, Sing’s also got a point, tau! Our KK8 room G101 always has Sing with brilliant ideas one lah!’

Kah Sing: ‘Hmm, why not I write a program to test our understanding of Roman numerals after class? Who knows, maybe it’d help us in our study for this module?’

The Roman numerals, originating from ancient Rome and widely used until the Late Middle Ages in Europe, employ letters from the Latin alphabet to represent numbers. Each letter has a fixed value, with the modern system using only seven letters: I, V, X, L, C, D, and M; each of which represents each of the integers 1, 5, 10, 50, 100, 500, 1000.

Many people misunderstand Roman numerals, thinking they always follow a subtractive syntax. In this system, a smaller digit before a larger one is subtracted from the larger one. However, Romans may opt for alternate systems instead (like what Kah Sing suggested)! Romans themselves only really used the subtractive syntax for smaller numbers, as it was largely avoided for larger numbers to give more clarity (or even for smaller numbers purely for simplicity, like what Ridwan and Suresh brought up).

Requirements

1. Under the class Romans,
 - a. Create a method named generateInitials to generate the initials of the user(s) of the computer program.

- b. Create a method named `convertArabics` to convert the given roman numeral (which only contains I, V, X, L, C, D, M) to Arabic form (which only contains 1, 2, 3, 4, 5, 6, 7, 8, 9, 0).
 - c. Create a method named `convertRomans` to convert the given Arabic numeral to roman form.
2. Ensure there is no error in compiling, running and handling invalid inputs in the code, and when sample inputs and outputs are passed in, there is no error found by the grader.
3. Ensure your report is done and everything you include in the report aligns with all the aspects of your code.

Solution

Firstly, we need to import Scanner class for user to input *nth* number of roman equations. We start the code with a java method call *generateInitials* with parameter *names* as a String object to display initial letters from usernames after user done input the equations. In this java method, the first line should display “Statements for the Roman numeral test are sent in by ” followed by a for loop statement. This for loop statement contains nested if statements which the first if statement will find the first letter of each names by checking the characters of the string one by one and if the character after a space between names, it is the first letter. Inside this if statement has if-else statements with the first if statement is to check if the first letter of the names start with uppercase or not using `Character.isUpperCase(names.charAt(i))`. If it starts with uppercase then it should print that letter as the initials, but if it is not an uppercase, than it not be printed as it is a delimiters(bin, binti, a/l, a/p, @). If it is a comma, then else-if statement should be executed which will print a comma instead. Outside this for loop, it will the full user name in a bracket that their input when the program starts.

Next, create another java method named *convertArabics* with parameter *romans* with datatype char to return int number according to romans numeral. In this java method, create a switch case statements with the case of romans characters. If the case is M, then it will return 1000, if the case is D, then it will return 500 and so on. If the case is not one of the romans numeral, then it will return 0 in default. After this method, create another method named *convertRomans* with parameter *number* with data type int. Declare two arrays, int array for store arabics number (1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1) and string array for romans numeral (“M” , "CM", "D", "CD",

"C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"). Declare a variable string named *roman*. Create a for loop and a while loop inside the for loop. This for loop is to check the *arabics* array one by one. The while loop inside it is to check if the *numbers* is greater than or equal to that current *arabics* index. If true, then string *roman* of the same index will be added.

Next, another java method are created to operate changing romans to arabics. This method named *RomantoArabic* with parameter *equation* as a String object and it should return the arabics number. The first line of this method should convert every String to uppercase by using *.toUpperCase()* function. This is because we set the letter default in uppercase in *convertArabics* method so the user can input romans equation in both lower and uppercase. Then declare *int sum = 0* as our final arabic numbers that will be return. Create a for loop and inside it declare *int first* and *int second*. Variable *first* will call *convertArabics* to convert one by one character in *equation* by using *.charAt(i)* and also the *second* will equal to the next character by call *convertArabics(equation.charAt(i+1))*. Then, create if-else statement that check the *first* is less than *second*. If true, then *sum* will equal to *sum* minus the *first*. If false, then else statement execute which *sum* is equal to *sum* plus the *first*. After done for loop, return *sum*.

Now, create main method. In main method, scan the String input that user will input their names and also scan the int input as a total equation that user want to input. If-else statement is added to check if total data was between 1 inclusive or 10 inclusive. If not, then print "Out of range." .If it is true, then the operation code will continue execute in else statement. In else statement, the user will repeatedly input romans equation with answer as a String *nth* time until it reach the total data. This input process will be executed in for loop as user input the equation repeatedly and the equation will be stored in String array named *romansEquation*. After for loop is done, we called the method *generateInitials* with usernames as the parameter to print their initials and full names. Declare a variable integer named *correct_statement* as 0 for further use. Create a for loop to execute the converting process to one by one String array equation. For each array index, the String equation will be split by 5 parts using function *.split(" ")* to separate romans numeral and operand. The arrangement is *romans_1 arithmetic_operand romans_2 equal_sign romans_3*. Declare a variable String for each *romans_numeral* and variable char for *arithmetic_operand* according to their parts(*romans_1 = parts[0]*, *romans_2 = parts[2]*). Then, declare three variables integer named *arabics_1* until 3. Each of this variable will equal to the method *RomantoArabic* with the same *romans* index as the parameter. This will convert the *romans_numeral* that in String to *arabics_number* in integer so we can start the calculation operation. After that, declare

correct_answer equal to 0. This variable is to find the real answer of the equation. Also declare boolean isOperand equal to true for default switch case. Create a switch statement for the arithmetic_operand. If the case is '+', then arabics_1 will plus arabics_2 and it will assign to correct_answer and it is the same for every operand. For default statement change isOperand equal to false. Next step is the validation process, create an if-else statement where the first if statement is to check if the arabics_3 and correct_answer is 1 inclusive and 3999 inclusive and also isOperand is not equal to true. If it so, print "Invalid Statement". If not, else if statement is executed to check if the correct answer is equal to arabics_3. If it so, then print arabics_1 arithmetic_operand arabics_2 equal_sign arabics_3 and append "is correct" and increment correct_statement variable by one. If not, execute other else statement which will append "is wrong, as the answer should be" and call the *convert Romans* method with correct_answer as the parameter to show the correct answer in romans and continue append with " which is " correct_answer " in Arabic numeral form."

Then, print one empty line. Declare a variable double called percentage to calculate the percentage of the correct_statement. Then, print "Number of correct statement = " append it with correct_statement. After that, print "Percentage of correct statement = " using printf to give the percentage in 2 decimal points and append it with variable percentage.

Pseudocode

1. Start
2. create *generateInitials(String names)* method
 - 2.1 Display “Statements for the Roman numeral test are sent in by”
 - 2.2 For loop
 - 2.2.1 for(int i =0; i<names.length(); i++)
 - 2.2.2 if (i==0 || names.charAt(i-1)==' ' || names.charAt(i-1)=='-' || names.charAt(i-1) == '.' || names.charAt(i-1)=='_')
 - 2.2.2.1 if (Character.isUpperCase(names.charAt(i))), display names.charAt(i)
 - 2.2.2.2 else if(names.charAt(i) == ','), display “ , ”
 - 2.3 Print “(” and print names and “) \n”
 - 2.4 Print one empty line
3. create *convertArabics(char romans)* method
 - 3.1 Switch case
 - 3.1.1 switch(romans)
 - 3.1.1.1 case ‘M’ : return 1000;
 - case ‘D’ : return 500;
 - case ‘C’ : return 100;
 - case ‘L’ : return 50;
 - case ‘X’ : return 10;
 - case ‘V’ : return 5;
 - case ‘I’ : return 1;
 - Default : return 0;
4. create *convertRomans(int number)* method
 - 4.1 Declare int array named arabics
 - 4.1.1 Store value 1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, and 1

- 4.2 Declare String array named romans
 - 4.2.1 Store value "M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX", "V", "IV", "I"
- 4.3 Declare variable String named roman
- 4.4 For loop
 - 4.4.1 for(int i=0; i<arabics.length; i++)
 - 4.4.2 While loop
 - 4.4.2.1 while(number>=arabics), roman+=romans[i], number-=arabics[i]
- 4.5 return roman
- 5 create *RomantoArabic*(String equation) method
 - 5.1 Declare equation=equation.toUpperCase()
 - 5.2 Declare variable int named sum = 0
 - 5.3 For loop
 - 5.3.1 for(int i = 0; i<equation.length();i++)
 - 5.3.2 declare 2 variables int named first and second
 - 5.3.3 set first equal to method *convertArabics*(equation.charAt(i))
 - 5.3.4 set second equal to method *convertArabics*(equation.charAt(i+1)), if i+1 is equal or greater than equation.length(), set second to 0 instead.
 - 5.3.5 If statement: if first<second, sum-=first
otherwise , sum+=first
 - 5.4 return sum
- 6 Main method
 - 6.1 Scan input usernames from user and store in variable String named users_name
 - 6.2 Scan input total data from user and store in variable named int named total_data

- 6.3 If-else statement: if total data less than 1 or greater than 10, print “Out of range!”
- 6.4 Else, declare a String array named romansEquation with the size of total data
 - 6.4.1 For loop: for(int i = 0; i<romansEquation.length; i++)
 - 6.4.1.1 Scan input roman equation from user and store in current index array
 - 6.4.2 Call *generateInitials*(users_name) method
 - 6.4.3 Declare a variable int named correct_statement = 0
 - 6.4.4 For loop: for(int i = 0; i<romansEquation.length; i++)
 - 6.4.4.1 Declare String array named parts equal to
romansEquation[i].split(“”)
 - 6.4.4.2 Declare String variable called romans_1, romans_2, romans_3, and
variable char named arithmetic_operand.
 - 6.4.4.3 Assign parts[0] to romans_1, parts[1].charAt(0) to a r
arithmetic_operand, parts[2] to romans_2, parts[3] to romans_3.
 - 6.4.4.4 Declare 3 variables int named arabics_1, arabics_2 and arabics_3
 - 6.4.4.5 Call method *RomantoArabic* with parameter romans_1 and assign
it to arabics_1. Do the same for arabics_2 and arabics_3 with the
same index for romans.
 - 6.4.4.6 Declare a variable int named correct_answer = 0
 - 6.4.4.7 Declare a variable boolean named isOperand and set the value to
true.
 - 6.4.4.8 Switch case: switch(arithmetic_operand)
 - 6.4.4.8.1 case ‘+’ : correct_answer=arabics_1 + arabics_2
break;
 - case ‘-’ : correct_answer=arabics_1 - arabics_2

```

break;

case '+' : correct_answer=arabics_1 * arabics_2

break;

case '/' : correct_answer=arabics_1 / arabics_2

break;

case '%' : correct_answer=arabics_1 % arabics_2

break;

case '^' : correct_answer=(int)Math.pow(arabics_1,
    arabics_2)

break;

default : isOperand = false

```

6.4.4.9 If statement: if(arabics_3<1 || arabics_3>3999 || correct_answer<1 || correct_answer>3999 || !isOperand), print “Invalid Statement”

Else if (correct_answer==arabics_3), print the equation in arabics and append “is correct.”. Then, increment correct_statement by one.

Else if(correct_answer != arabics_3), then print the equation in arabics and append “is wrong, as the answer should be” and call the *convertRomans*(correct_answer) method and append with “which is” and add correct_answer and append with “in Arabics numeral form.”

6.4.5 Print one empty line

6.4.6 Declare variable double named percentage =
(correct_statement/(double)total_data)*100

6.4.7 Print “Number of Correct Statement = ” and append correct_statement

6.4.8 Print “Percentage of Correct Statement = ” and append percentage(using printf to get 2 decimal point)

7 End

Sample input and output

```
Ahmad Sani bin Albab , Suresh a/l Pallavarajan , Lee Chong Wei
5
XXIV + MMCD = MMCDXXIV
Mm / cD = v
V ^ V = MMMxxv
MDCLXVI = IVXLCDM + MCCCXXXII
V - I = Iv
Statements for the Roman numeral test are sent in by ASA , SP , LCW (Ahmad Sani bin Albab , Suresh a/l Pallavarajan , Lee Chong Wei)

24 + 2400 = 2424 is correct.
2000 / 400 = 5 is correct.
5 ^ 5 = 3025 is wrong, as the answer should be MMMCXXV which is 3125 in Arabic numeral form.
Invalid Statement.
5 - 1 = 4 is correct.

Number of Correct Statement = 3
Percentage of Correct Statements = 60.00%
```

Source Code

```
import java.util.Scanner;

public class Romans{

    static void generateInitials(String names){

        System.out.print("Statements for the Roman numeral test are sent in by
");
        for(int i = 0; i < names.length(); i++){

            if(i == 0 || names.charAt(i-1) == ' ' || names.charAt(i-1) == '-' ||
names.charAt(i-1) == '_' || names.charAt(i-1) == '.'){

                if(Character.isUpperCase(names.charAt(i))){

                    System.out.print(names.charAt(i));
                }else if(names.charAt(i) == ','){

                    System.out.print(" , ");
                }
            }
        }
        System.out.print(" (" + names + ") \n");
        System.out.println();
    }

    static int convertArabics(char romans){

        switch(romans){
            case 'M' :
                return 1000;
            case 'D' :
```

```

        return 500;
    case 'C' :
        return 100;
    case 'L' :
        return 50;
    case 'X' :
        return 10;
    case 'V' :
        return 5;
    case 'I' :
        return 1;
    default :
        return 0;
    }
}

static String convertRomans(int number){

    int[] arabics = {1000, 900, 500, 400, 100, 90, 50, 40, 10, 9, 5, 4, 1};
    String[] romans = {"M", "CM", "D", "CD", "C", "XC", "L", "XL", "X", "IX",
        "V", "IV", "I"};

    String roman = "";
    for(int i = 0; i < arabics.length; i++){

        while(number >= arabics[i]){

            roman += romans[i];
            number -= arabics[i];
        }
    }
    return roman;
}

```

```

static int RomantoArabic(String equation){

    equation = equation.toUpperCase();
    int sum = 0;
    for(int i = 0; i < equation.length(); i++){

        int first = convertArabics(equation.charAt(i));
        int second = (i+1 < equation.length()?
convertArabics(equation.charAt(i+1)) : 0);

        if(first < second){
            sum -= first;
        }else{
            sum += first;
        }
    }
    return sum;
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    String users_name = sc.nextLine();
    int total_data = sc.nextInt();

    if(total_data < 1 || total_data > 10){
        System.out.println("Out of range!");
    }else{
        //input roman equations
        sc.nextLine();
        String[] romansEquation = new String[total_data];
        for(int i = 0; i < romansEquation.length; i++){

```

```

        romansEquation[i] = sc.nextLine();
    }

    generateInitials(users_name);

    int correct_statement = 0;

    for(int i = 0; i < romansEquation.length; i++){

        String[] parts = romansEquation[i].split(" ");
        String romans_1 = parts[0];
        char arithmetic_operand = parts[1].charAt(0);
        String romans_2 = parts[2];
        String romans_3 = parts[4];

        //convert Roman to Arabic
        int arabics_1 = RomantoArabic(romans_1);
        int arabics_2 = RomantoArabic(romans_2);
        int arabics_3 = RomantoArabic(romans_3);
        // find operand
        int correct_answer = 0;
        boolean isOperand = true;
        switch(arithmetic_operand){
            case '+' :
                correct_answer = arabics_1 + arabics_2;
                break;
            case '-' :
                correct_answer = arabics_1 - arabics_2;
                break;
            case '*' :
                correct_answer = arabics_1 * arabics_2;
                break;

```



```

        case '/' :
            correct_answer = arabics_1 / arabics_2;
            break;
        case '%' :
            correct_answer = arabics_1 % arabics_2;
            break;
        case '^' :
            correct_answer = (int)Math.pow(arabics_1, arabics_2);
            break;
        default :
            isOperand = false;
    }

    //validation
    if((arabics_3 < 1 || arabics_3 > 3999) || (correct_answer < 1 ||
correct_answer > 3999) || !isOperand){

        System.out.println("Invalid Statement.");
    }else if(correct_answer == arabics_3){

        System.out.println(arabics_1 + " " + arithmetic_operand + " " +
arabics_2 + " = " + arabics_3 + " is correct.");
        correct_statement++;
    }else if(correct_answer != arabics_3){

        System.out.println(arabics_1 + " " + arithmetic_operand + " " +
arabics_2 + " = " + arabics_3 + " is wrong, as the answer should be " +
convertRomans(correct_answer) + " which is " + correct_answer + " in Arabic
numeral form.");
    }

}

System.out.println();

```

```
        double percentage = (correct_statement/(double)total_data)*100;
        System.out.println("Number of Correct Statement = " +
correct_statement);
        System.out.printf("Percentage of Correct Statements = %.2f%%",
percentage);
    }
}
}
```

QUESTION 4

Problem Statement

Write a function called `format` that takes an integer number and an integer width as input. The function should return a string representation of the number, padded with zeros on the left to reach the specified width. If the number is already longer than the width, the function should return the original string representation of the number. Here's an example of how the function should work:

- `format(34, 4)` should return `"0034"`.
- `format(34, 5)` should return `"00034"`.
- `format(34, 1)` should return `"34"`.

Then, write a program that prompts the user to enter an integer and a width, then calls the `format` function and displays the resulting string.

Solution

The `Scanner` class is imported to allow the program to read user input. A `Scanner` variable, `in`, is created to read inputs and two integers, `num` and `width`, are declared to store the integer inputted and the length of the integer respectively. The program then prompts the user with `"Format :"` and the user will input the integer (`num`) and its length (`width`). The `format` method is created with `num` and `width` as arguments. In the `format` method, the integer `x` is converted to a string (`fString`) to calculate its length (`size`). If the length of the number (`size`) is greater than or equal to the specified width (`y`), the method prints the number as is. But if the number's length is less than the specified width, the program will calculate the number of leading zeros needed (`y - size`), print the zeros using a `for` loop and finally, print the original number as `fString`.

Pseudocode

1. Start
2. Import Scanner into the java file
3. Declare in as new Scanner
4. Declare num and width as integer
5. Print “Format : ”
6. Read num and width as integer input
7. Call the method format
8. Method format(int x, int y)
 - 8.1. Initialize fString as string by converting integer x to string
 - 8.2. Initialize size as length of fString
 - 8.3. If size is greater than or equal to y
 - 8.3.1. Print “Formatted integer : ” and fString
 - 8.4. Else
 - 8.4.1. Print “Formatted integer : ”
 - 8.4.2. For i less than y-size
 - 8.4.2.1. Print “0”
 - 8.4.3. Print fString
 - 8.5. Return 0
9. End

Sample input and output

Format : 34 5
Formatted integer : 00034

Format : 869 2
Formatted integer : 869

Format : 4000 4
Formatted integer : 4000

Format : 1869 9
Formatted integer : 000001869

Format : 324 0
Formatted integer : 324

Source Code

```
import java.util.Scanner;

public class V2Q4 {

    public static void main(String[] args) {

        Scanner in=new Scanner(System.in);

        int num, width;

        System.out.print("Format : ");

        num=in.nextInt();

        width=in.nextInt();

        format(num, width);

    }

    private static int format(int x, int y){

        String fString=Integer.toString(x);

        int size=fString.length();

        if(size>=y){

            System.out.println("Formatted integer : "+fString);

        }

        else{

            System.out.print("Formatted integer : ");

            for(int i=0;i<(y-size);i++){

                System.out.print("0");

            }

            System.out.print(fString);

        }

    }

}
```

```
        return 0;  
    }  
}
```

QUESTION 5

Problem

Write a Java program that implements a billing system for a fruit store, FreshMart. The program will display the available fruits, take user input to select a fruit and quantity, check the stock availability, and generate a bill. You are provided the main method, which handles user input and basic program flow. Your job is to complete the remaining methods so that the program behaves as expected.

We are provided the main method, which handles user input and basic program flow. Your job is to complete the remaining methods so that the program behaves as expected.

```
import java.util.Scanner;

public class freshMart {
```

```
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        String[] fruits = {"Apple", "Grape", "Banana", "Mango", "Orange", "Strawberry",
"Blueberry"};
        int[] storage = {45, 57, 75, 34, 75, 65, 78};
        double[] price = {5.00, 10.00, 2.50, 6.00, 3.00, 15.00, 12.00};

        printFruit(fruits);

        System.out.print("Please enter the fruit you want to buy: ");
        String fruitSelect = in.nextLine();

        System.out.print("Unit or Box: ");
        int quantity = in.nextInt();

        int fruitIdx = getIdxFruit(fruits, fruitSelect);

        if (!haveProduct(fruits, fruitIdx)) {
            System.out.println("Sorry we currently do not offer this product.");
        } else if (!haveStock(fruitIdx, storage, quantity)) {
            System.out.println("Sorry, we do not have enough stock for " + fruits[fruitIdx]
+ ". Please come again later!");
        } else {
            System.out.println("\nKindly Checkout here: ");
            generateBill(fruits, fruitIdx, price, quantity);
        }
    }
}
```

1. `printFruit(String[] fruits)`: This method should display the list of available fruits in the store.

2. `getIdxFruit(String[] fruits, String selectedFruit)`: This method should return the index of the selected fruit in the array. If the fruit is not found, it should return -1.
3. `haveProduct(String[] fruits, int fruitIdx)`: This method checks if the selected fruit is available in the store.
4. `haveStock(int fruitIdx, int[] storage, int quantity)`: This method checks if the selected fruit has enough stock to meet the customer's requested quantity.
5. `generateBill(String[] fruits, int fruitIdx, double[] price, int quantity)`: This method generates and prints the bill, showing the product, quantity, price per unit, and total price.

Solution

- Firstly, we must import the Scanner class to handle user input
- Secondly, a `printFruit` method is implemented to display the available fruits. In the method, there is a for loop that will loop through each index of the fruit array. After that, each fruit with its index(i+1) will be printed.
- Next, a `getIdxFruit` method is implemented. In the method, a for loop will loop through each index of the fruit array. Then, in the if statement, it compares `selectedFruit`(user input) to each fruit in the array. If it matches, the program will return the index of the fruit, or returns -1 if the fruit is not in the list.
- After that, a `haveProduct` method which is a boolean type is implemented. It will check whether `fruitIdx` is in the range of 0 to `fruit.length -1`. If it is in the range, returns true otherwise returns false.
- Then, a `haveStock` method which is also a boolean type is implemented. It will check if the stock of the selected fruit is greater or equal to the requested quantity of the user. If the stock is sufficient, returns true, return false otherwise.
- Lastly, a `generateBill` method is implemented. This method will print a receipt showing the selected fruit name, quantity purchased, price per unit and total price.
- In the main method, we must initialize the fruits array with the list of available fruits, the storage array with stock quantities for each fruit and the price array with prices per unit of each fruit.
- We call the `printFruit(fruits)` to display all the list of the fruits. Prompts the user to enter the fruit name (`fruitSelect`). Prompts the user to enter the quantity they want to purchase (`quantity`).

- After that, we must call `getIdxFruit` to find the index of the selected fruit and assign it to `fruitIdx`.
- Entering the if statement, the first check is if the `haveProduct` method returns false, it will print “Sorry we currently do not offer this product.”. the second check is if the quantity that the user input is greater than the available stock, the `haveStock` method will return false and it will print “Sorry, we do not have enough stock for “ + `fruits[fruitIdx]` + “. Please come again later!”. Else it will call the `generateBill` method to print a receipt.

Pseudocode

printFruit (Method)

1. Print “Freshmart Fruit Selection” .
2. Loop through the fruits array.
 - 2.1. print the fruit index(i+1) and the name of the fruit.

getIdxFruit (Method)

1. loop through each fruit array.
 - 1.1 if the current fruit matches the selected fruit(user input).
 - 1.2 return the index of the fruit.
2. if the current fruit doesn't match the selected fruit.
 - 2.1 return -1.

haveProduct (Method)(boolean)

1. Check whether fruitIdx is in the range of 0 to fruit.length -1.
2. return true if it matches the condition .
3. return false if otherwise.

haveStock (Method)(boolean)

1. Check if the stock of the fruit selected(storage[fruitIdx]) is greater or equal to the quantity(user input).
2. return true if there is enough stock of the fruit .
3. return false otherwise.

generateBills (Method)

1. Calculate totalPrice=price[fruitIdx]*quantity.
2. Print a receipt.
 - 2.1 Header: "Fresh Mart Receipt".

2.2 Product name, quantity, and price per unit.

2.3 Total price.

main (Method)

1. Initialize String[] fruits = {"Apple", "Grape", "Banana", "Mango", "Orange", "Strawberry", "Blueberry"};
2. Initialize int[] storage = {45, 57, 75, 34, 75, 65, 78};
3. Initialize double[] price = {5.00, 10.00, 2.50, 6.00, 3.00, 15.00, 12.00};
4. Call printFruits(fruits) to display available fruits .
5. Prompt the user to enter the fruit name (fruitSelect).
6. Prompt the user to enter the quantity of the fruit that they want (quantity).
7. Call getIdxFruit(fruits, fruitSelect) and assign the index of the selected fruit to fruitIdx.
8. if haveProduct(fruits, fruitIdx) returns false
 - 8.1 Print “Sorry we currently do not offer this product.”
 - 8.1.1 else if haveStock(fruitIdx, storage, quantity) returns false
 - 8.1.2 Print “Sorry, we do not have enough stock for the fruit name. Please come again later!”
 - 8.1.3 else print “Kindly Checkout here.”
 - 8.1.4 Call generateBill(fruit, fruitIdx, price, quantity

Sample Input and Output

1.

```
Freshmart Fruit Selection
1. Apple
2. Grape
3. Banana
4. Mango
5. Orange
6. Strawberry
7. Blueberry
Please enter the fruit you want to buy: orange
Unit or Box: 11
```

```
Kindly Checkout here:
```

```
-----
-----Fresh Mart Receipt-----
-----
Product           Quantity      Price per Unit(RM)
Orange             11              3.00
-----
Total Price: RM33.00
BUILD SUCCESSFUL (total time: 3 seconds)
```

2.

```
Freshmart Fruit Selection
1. Apple
2. Grape
3. Banana
4. Mango
5. Orange
6. Strawberry
7. Blueberry
Please enter the fruit you want to buy: Apple
Unit or Box: 47
Sorry, we do not have enough stock for Apple. Please come again later!
BUILD SUCCESSFUL (total time: 9 seconds)
```

3.

```
Freshmart Fruit Selection
1. Apple
2. Grape
3. Banana
4. Mango
5. Orange
6. Strawberry
7. Blueberry
Please enter the fruit you want to buy: guava
Unit or Box: 6
Sorry we currently do not offer this product.
BUILD SUCCESSFUL (total time: 5 seconds)
```

4.

```
Freshmart Fruit Selection
1. Apple
2. Grape
3. Banana
4. Mango
5. Orange
6. Strawberry
7. Blueberry
Please enter the fruit you want to buy: 4
Unit or Box: 5
Sorry we currently do not offer this product.
BUILD SUCCESSFUL (total time: 4 seconds)
```

Source Code

```
package freshmart;

import java.util.Scanner;

public class freshMart {

    public static void printFruit(String[] fruits) {

        System.out.println("Freshmart Fruit Selection");

        for (int i=0;i<fruits.length;i++) {

            System.out.println((i+1)+". "+fruits[i]);

        }

    }

    public static int getIdxFruit(String[] fruits,String selectedFruit){

        for(int i=0;i<fruits.length;i++){

            if(fruits[i].equalsIgnoreCase(selectedFruit)){

                return i;

            }

        }

        return -1;

    }

    public static boolean haveProduct(String[] fruits,int fruitIdx){

        return fruitIdx>=0&&fruitIdx<fruits.length;

    }

    public static boolean haveStock(int fruitIdx,int[] storage,int quantity){

        return storage[fruitIdx]>=quantity;

    }

}
```

```

    }

    public static void generateBill(String[] fruits,int fruitIdx,double[] price,int quantity){

        double totalPrice=price[fruitIdx]*quantity;

        System.out.println("-----");

        System.out.println("-----Fresh Mart Receipt-----");

        System.out.println("-----");

        System.out.println("Product      Quantity   Price per Unit(RM)");

        System.out.printf("%-12s %13d %22.2f%n",fruits[fruitIdx],quantity,price[fruitIdx]);

        System.out.println("-----");

        System.out.printf("Total Price: RM%.2f",totalPrice);

        System.out.println("");

    }

    public static void main(String[] args) {

        Scanner in = new Scanner(System.in);

        String[] fruits = {"Apple", "Grape", "Banana", "Mango", "Orange", "Strawberry",
"Blueberry"};

        int[] storage = {45, 57, 75, 34, 75, 65, 78};

        double[] price = {5.00, 10.00, 2.50, 6.00, 3.00, 15.00, 12.00};

        printFruit(fruits);

        System.out.print("Please enter the fruit you want to buy: ");

        String fruitSelect = in.nextLine();

        System.out.print("Unit or Box: ");

        int quantity = in.nextInt();

        int fruitIdx = getIdxFruit(fruits, fruitSelect);

```



```
if (!haveProduct(fruits, fruitIdx)) {  
    System.out.println("Sorry we currently do not offer this product.");  
} else if (!haveStock(fruitIdx, storage, quantity)) {  
    System.out.println("Sorry, we do not have enough stock for " + fruits[fruitIdx]  
        + ". Please come again later!");  
} else {  
    System.out.println("\nKindly Checkout here: ");  
    generateBill(fruits, fruitIdx, price, quantity);  
}  
  
}  
  
}
```

QUESTION 6

Problem

In this problem, I write code that allows the user to do the following:

- i Take input of $n \times n$ integers to form a matrix (2D array)
- ii Rotate the matrix by 90 degrees clockwise
- iii Display the original matrix and the matrix after rotating it

Implementation:

Method 1: *parseMatrix*

- a) A method called ***parseMatrix*** will prompts the user to enter an integer ***n***. This will be the dimensions of the $n \times n$ matrix.
- b) Next, prompt the user to enter the ***n*** rows of the matrix each with ***n*** integers to form the matrix.
- c) Store the input matrix as a 2D array for further processing
- d) It is given that:
 - i) The user will always input an integer
 - ii) The user will only enter exactly $n \times n$ integers in total
 - iii) The user will input number starting from first row, from left to right
 - iv) ***n is a number between 0 and 100***

Method 2: *displayMatrix*

- a) A method called ***displayMatrix*** that takes a 2D array as input and outputs the array in the form of a matrix (refer to the sample input and output for examples).
- b) Use this method to display the input matrix we obtained from the ***parseMatrix*** method and to ensure that your ***parseMatrix*** method correctly retrieves the matrix.

Method 3: *rotateMatrix*

- a) A Java method ***rotateMatrix*** that takes a 2D array (matrix) as input and rotates the matrix 90 degrees clockwise. The method rotate the matrix in place without returning anything.
- b) This method rotate the input matrix and then use the ***displayMatrix*** method to display the matrix after it has been rotated.

Solution

- 1) First method : inputting the matrix
 - a) The function will be an `int[][]` function that return 2D array values.
 - b) We check the input dimension by the user. It should be between 0 and 100.
 - c) By using for loop, we loop the program for inputting the value between each row and column.
- 2) Second method : display the matrix
 - a) The function will be a void type function.
 - b) By using for loop, we can display the value for the matrix.
 - c) To determine the length of the matrix, we can use `array.length()` function.
- 3) Third method : rotating the matrix
 - a) The function will be a void type function.
 - b) To rotate the matrix, we need to do a 2 nested for loop.
 - c) By using `rotatedMatrix[i][j]=matrix[n-j-1][i]`; we can put the respective value inside a new matrix.
 - d) Later we use the second method to display the matrix.

Pseudocode

Main method :

1. Define a matrix without dimension
2. Call function parse matrix into variable matrix introduced before
3. Print out matrix
4. Transpose the matrix using function rotatematrix

Parse matrix

1. User input the dimension
2. Check whether the dimension between the range of 0-100
3. Define a matrix with the size of the dimension stated by the user
4. Using a nested for loop to loop the input from the user.
5. Return the matrix variable

Display matrix

1. Use array.length to determine the size of the array
2. By using nested for loop, print out the value of the matrix one by one

Rotate matrix

1. Use array.length to determine the size of the array
2. Define a new matrix named rotatedmatrix with the size of the array.length
3. By using nested for loop, rotate the matrix using program :
 rotatedMatrix[i][j]=matrix[n-j-1][i];
4. Call displaymatrix method to display the rotated matrix

Sample input and output

```
run:
Enter the dimension of the square matrix : 3
Enter row number 1 : 1 2 3
Enter row number 2 : 4 5 6
Enter row number 3 : 7 8 9
The original matrix
1   2   3
4   5   6
7   8   9
The rotated matrix :
7   4   1
8   5   2
9   6   3
BUILD SUCCESSFUL (total time: 15 seconds)
```

```
run:
Enter the dimension of the square matrix : 5
Enter row number 1 : 1 2 3 4 5
Enter row number 2 : 6 7 8 9 10
Enter row number 3 : 11 12 13 14 15
Enter row number 4 : 16 17 18 19 20
Enter row number 5 : 21 22 23 24 25
The original matrix
1   2   3   4   5
6   7   8   9   10
11  12  13  14  15
16  17  18  19  20
21  22  23  24  25
The rotated matrix :
21  16  11  6   1
22  17  12  7   2
23  18  13  8   3
24  19  14  9   4
25  20  15  10  5
BUILD SUCCESSFUL (total time: 34 seconds)
```

Source code

```
import java.util.Scanner;
public class Q6 {
    public static void main(String[] args) {
        int[][] matrix;
        matrix = parseMatrix();
        System.out.println("The original matrix");
        displayMatrix(matrix);
        rotateMatrix(matrix);
    }
    public static int[][] parseMatrix(){
        Scanner scanner = new Scanner(System.in);
        int n=0;

        //user input the dimension
        System.out.print("Enter the dimension of the square matrix : ");
        do{
            n = scanner.nextInt();
            if(n>0 && n<=100)
                break;
            else
                System.out.print("Please enter a dimension between 0 and 100 : ");
        }while(true);
        int[][] matrix= new int[n][n];

        //input the matrix value
        for(int i=0; i<n; i++){
            System.out.print("Enter row number "+(i+1)+" : ");
            for(int j=0; j<n; j++){
                matrix[i][j] = scanner.nextInt();
            }
        }
    }
}
```

```

        return matrix;
    }

    public static void displayMatrix(int[][] matrix){
        for(int[] arr : matrix){
            for(int value : arr){
                System.out.printf("%-4s",value);
            }
            System.out.println("");
        }
    }

    public static void rotateMatrix(int[][] matrix){
        int n = matrix[0].length;
        int[][] rotatedMatrix = new int[n][n];

        for(int i=0; i<n; i++){
            for(int j=0; j<n; j++){
                rotatedMatrix[i][j]=matrix[n-j-1][i];
            }
        }
        System.out.println("The rotated matrix :");
        displayMatrix(rotatedMatrix);
    }
}

```